2. b. Memory Efficient Doubly Linked List

–Creation of Empty List – Space Complexity and Algorithm

Program:

```
#include <cstdint> // Required for uintptr_t (essential for
pointer XOR)
#include <cstdlib> // Required for malloc and free
#include <iostream>

typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;

Node *head;

Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}

void createEmptyList()
{
    head = nullptr;
}

int main()
{
    createEmptyList();
    return 0;
}
```

Space Complexity

XOR Linked List Analysis

1. ***Input Space Complexity*** **(*Parameter − Only*):** ***This measures the space required by the parameters passed to the functions.***

- ***XOR*****(*a, b*):** ***Takes two pointers as parameters. Each pointer occupies a fixed amount of memory*** **(4 *or* 8 *bytes*).**
- ***createEmptyList*****():** ***Takes no parameters.***
- ***main*****():** ***Takes no parameters.***
- ***Conclusion*****:** ***The Input Space Complexity*** **(*Parameter − Only*) *is O*(1).**

2. ***Input Space Complexity*** **(*Full − Data*):** ***This considers the space occupied by the data structure upon which the program operates.***

- ***Global Variable*** **(*head*):** ***The program declares a single global pointer head. A pointer occupies a fixed size.***
- ***The List Nodes*****:** ***At this specific stage*** **(*initialization*), *the list is empty. There are n* = 0 *nodes.***
- ***Analysis*****:** ***Since the list is currently empty, the only "data" present is the global head pointer.***

- ***Conclusion*: *The Input Space Complexity* (*Full − Data*) *is* $O(1)$.**

3. *Auxiliary Space Complexity*

***This measures the extra space used during the execution of the functions* (*stack space, local variables, dynamic allocation*).**

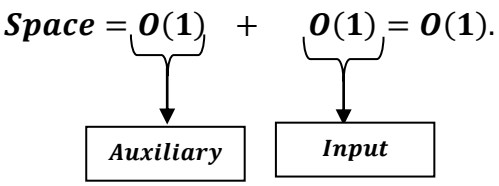
- ***Variables*: $*$ *createEmptyList uses no local variables*.**
    - ***XOR uses type casting internally to perform the bitwise operation, which happens in registers or a tiny amount of stack space*.**
- ***Dynamic Allocation*: *There is no use of malloc or new in this specific snippet*.**
- ***Stack*: *The function call overhead for createEmptyList is constant. Even if XOR is called, it does not use recursion*.**
- ***Conclusion*: *The Auxiliary Space Complexity is* $O(1)$.**

Total Space Complexity

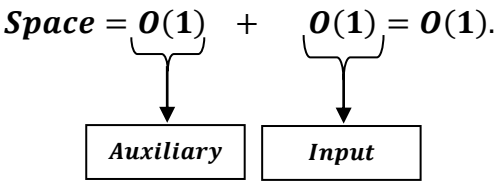
Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**



- **Holistic Interpretation (Full-data input):**



In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

Algorithm for Creation of Empty Memory Efficient Doubly Linked List

- ***Step 1: DEFINE STRUCTURE Define a structure DLL with three members:***
  - ***An integer data to store the value of the node.***
  - ***A pointer next of type DLL* to point to the next node in the list.***
  - ***A pointer prev of type DLL* to point to the previous node in the list.***
- ***Step 2: TYPE ALIAS: Use typedef to create an alias Node for struct DLL to simplify the syntax throughout the program.***
- ***Step 3: DECLARE GLOBAL POINTER: Declare a global pointer head of type Node*. This will act as the entry point to the list and allow all functions to access the list's starting position.***
- ***Step 4: CREATE INITIALIZATION FUNCTION: Define a function createEmptyList that takes no arguments:***
  - ***SET head := nullptr (or NULL in C) to indicate that the list currently contains no nodes.***
- ***Step 5: EXECUTE INITIALIZATION: In the main function, call createEmptyList() to initialize the data structure before any nodes are added.***